Resultados do experimento de vazamento

Projeto ETL Mercatto · Mineração de Dados · IFMT 2026/2 · Gabriel Lemos Gomes, Lucas Peres de Lima e Marcus Vinícius Santos de Almeida

Todo número deste relatório vem de arquivos gravados em `saida/experimento/` pelos scripts `etl/diagnostico/experimento_python.py` e `etl/diagnostico/experimento_sensibilidade.py`, e do `metricas_orange.csv` (copiado do Test & Score do Orange). A fonte de cada tabela está indicada abaixo dela.

1. O que foi comparado

- **Tabela-base.** 109.499 clientes, 21,96 % com recompra nos 90 dias seguintes ao corte de 30/06/2025 (`saida/abt/resumo_correta.csv`).
- **Correta:** todo atributo calculado só com dados até 30/06/2025.
- **Com vazamento:** `recencia_dias` e `frequencia_pedidos` calculados sobre a base inteira (inclusive os pedidos da janela do alvo) e `valor_monetario` padronizado (z-score) com média e desvio de treino e teste juntos.
- **Vazamento sutil:** só o z-score global, sem os outros dois atributos. Não foi ao Orange (regra do roteiro); só o Python a avaliou.
- **Controle:** versão correta com o z-score calculado só no treino.
- **Partição.** A mesma nos dois fluxos do Orange e no Python: 76.650 clientes no treino (16.830 positivos) e 32.849 no teste (7.212 positivos), estratificada, exportada do Orange (`particao_orange_*.csv`). O Python lê essa partição (hashes em `numeros.json`).
- **Modelos e avaliação.** Naive Bayes, Logistic Regression e Tree, com os parâmetros padrão do Orange 3.40.0, avaliados no conjunto de teste, com a classe 1 como positiva e limiar 0,5. Acurácia balanceada = (recall + especificidade) / 2, porque o Orange não a traz pronta e a acurácia simples (0,78 só prevendo "não") premia o modelo que nunca prevê recompra.
- **Dependências.** Para o Python, foram acrescentados `scikit-learn==1.5.2` e `scipy==1.14.1` a `etl/requirements.txt` e o contêiner `etl` foi reconstruído (a primeira tentativa falhou por falta de internet; a segunda construiu a imagem). Nenhum pacote foi instalado no host.

2. Resultados

Tabela 1: Orange (Test & Score, `metricas_orange.csv`; diferenças em `efeito_vazamento.csv`).

Modelo	AUC correta	AUC vazamento	Δ AUC	Ac. balanceada correta	Ac. balanceada vazamento	Δ
Naive Bayes	0,547	0,876	+0,329	0,520	0,741	+0,221
Logistic Regression	0,548	0,871	+0,323	0,500	0,6535	+0,1535
Tree	0,502	0,965	+0,463	0,507	0,952	+0,445

Tabela 2: Python, mesma partição (`metricas_python.csv`, `efeito_vazamento.csv`).

Modelo	AUC correta	AUC vazamento	Δ AUC	Ac. balanceada correta	Ac. balanceada vazamento	Δ
Naive Bayes	0,547	0,873	+0,326	0,522	0,751	+0,230
Logistic Regression (100 iterações)	0,526	0,847	+0,321	0,500	0,613	+0,113
Logistic Regression (padronizada)	0,549	0,872	+0,324	0,500	0,658	+0,158
Tree	0,505	0,959	+0,454	0,504	0,948	+0,444

Concordância entre Python e Orange (`comparacao_orange.csv`). Naive Bayes: diferença de AUC de no máximo 0,004 e de acurácia balanceada de no máximo 0,011. Tree: AUC a no máximo 0,006, mas recall e acurácia da versão correta diferem (recall 0,211 no Python contra 0,149 no Orange). Logistic Regression: AUC 0,022 e 0,024 abaixo do Orange. Explicações, que são hipóteses sustentadas pelo `sensibilidade.csv`, não confirmações:

- *Logistic Regression.* Com os padrões do Orange (100 iterações, atributos sem normalização), o scikit-learn do contêiner não converge. Com a otimização convergida, a AUC vai a 0,549 (correta) e 0,872 (vazamento), praticamente iguais às do Orange (0,548 e 0,871). A versão do scikit-learn do Orange instalado é mais nova que a do contêiner; não rodamos o Orange para testar se isso explica a convergência.
- *Tree.* O Orange para a divisão quando 95 % dos exemplos do nó são da mesma classe; o scikit-learn não tem esse critério e cresce até 12.087 folhas na versão correta. No `sensibilidade.csv`, o recall da árvore na versão correta cai de 0,224 (folha mínima de 1) para 0,010 (folha mínima de 100). O recall da árvore depende muito desse parâmetro; a AUC, bem menos.

3. Pergunta (a): diferença entre as versões em AUC e acurácia balanceada

Com o mesmo fluxo, os mesmos modelos e a mesma partição, trocar só o arquivo de entrada elevou a AUC de cerca de 0,5 para 0,87–0,97 nos três modelos, no Orange (+0,323 a +0,463, Tabela 1) e no Python (+0,321 a +0,454, Tabela 2). A acurácia balanceada subiu de 0,500–0,520 para 0,653–0,952 no Orange. O efeito é maior na Tree (+0,445 no Orange), que consegue usar `recencia_dias` e `frequencia_pedidos` para dividir os dados em poucos passos (1.133 folhas na versão com vazamento, contra 12.087 na correta).

Leitura: a versão correta quase não prevê (AUC próxima de 0,5, recall de 0,000 a 0,149 no Orange); a versão com vazamento parece excelente porque esses dois atributos já contêm informação sobre a janela do alvo. O número alto não mede capacidade de predição.

4. Pergunta (b): o vazamento sutil (normalização antes da partição) produziu diferença perceptível?

Não, nos números medidos. Só o Python avaliou essa versão (`efeito_vazamento.csv`, comparação "vazamento_sutil − correta"):

Modelo	AUC correta	AUC sutil	AUC controle (z só no treino)	Δ AUC sutil − correta
Naive Bayes	0,547	0,547	0,547	0,000
Tree	0,505	0,505	0,503	0,000
Logistic Regression (100 iterações)	0,526	0,540	0,538	+0,014
Logistic Regression (padronizada)	0,549	0,549	0,549	0,000

- A diferença de acurácia balanceada entre sutil e correta é no máximo 0,0003 em módulo, nos quatro modelos.
- O único modelo em que a AUC se mexe é a Logistic Regression sem convergência (+0,014). Mas o controle, que **não tem** vazamento, sobe quase o mesmo (+0,011), e quando a otimização converge as três versões dão a mesma AUC. Essa variação vem da otimização (a escala do atributo muda o caminho do algoritmo), não do vazamento.
- Isso é coerente com a natureza da transformação (uma mudança linear de escala não altera as faixas de mesma frequência do Naive Bayes nem as divisões da árvore), mas esta é uma explicação, não um resultado medido.

Conclusão: o vazamento sutil **existe** (as asserções de `04_abt.py --vazamento-sutil` confirmam que média e desvio vêm de todas as linhas), mas **não deixou rastro nas métricas** destes três modelos.

Ausência de efeito na métrica não é ausência de vazamento; ele só se descobre lendo o código ou a linhagem do dado, como na pergunta (c).

5. Pergunta (c): como descobrir que o resultado com vazamento está errado, sem saber como foi construído

Procedimentos, do mais barato ao mais caro. Os dois primeiros foram executados; os demais são o que faríamos e não rodamos.

1. **Plausibilidade (executado).** Uma tabela só de histórico de compras (recência, frequência, valor) com AUC de 0,959 numa Tree é suspeita por si só. O modelo de maioria dá acurácia de 0,78 (taxa de positivos 21,96 %), e na versão correta nenhum modelo passa de 0,549 de AUC. Resultado muito acima do que o problema costuma permitir pede auditoria antes de comemoração.
2. **AUC de cada atributo isolado (executado, `auc_univariada.csv`).** Na versão com vazamento, `recencia_dias` sozinho tem AUC 0,797 e `frequencia_pedidos` 0,708; os demais ficam em no máximo 0,536 (`ambos_sistemas`). Na versão correta nenhum passa de 0,536. Dois atributos concentrando quase todo o poder são o alvo da pergunta: "esse valor existia no momento da predição?".
3. **Auditoria de linhagem (não executada sobre o resultado, mas é o que fizemos na construção da versão correta).** Reconstruir cada atributo com o DW "como estava" na data de corte e comparar com o valor usado; atributos que mudam usam informação posterior.
4. **Validação fora do tempo (não executada).** Treinar com um corte anterior e testar num corte posterior: o vazamento se desfaz quando o atributo deixa de ser calculado com dados do futuro, e a AUC cai.
5. **Ablação (não executada).** Retirar os atributos suspeitos e ver o desempenho colapsar. O `auc_univariada.csv` é a evidência mais próxima que temos.

O vazamento sutil mostra o limite: nenhum desses procedimentos o acusaria, porque as métricas não mudam (seção 4). Só a revisão do código de preparação (a ordem entre normalizar e particionar) o revela.

6. Limitações

- O Orange foi rodado uma vez por versão, sem semente exposta para os modelos; os números do Python não o reproduzem ao centésimo (seção 2). As conclusões (a) e (b) valem nos dois e não dependem dessa diferença.
- A versão sutil e o controle não passaram pelo Orange.
- A qualidade preditiva da versão correta é baixa (AUC próxima de 0,5). O experimento mede o efeito do vazamento, não busca um bom modelo.

7. Reprodução

```
docker compose exec etl python 04_abt.py --correta --vazamento --vazamento-sutil
docker compose exec etl python 05_exportar_orange.py
docker compose exec etl python -m diagnostico.experimento_python        # ~20 s
docker compose exec etl python -m diagnostico.experimento_sensibilidade  # ~5 min
```

Arquivos: `saida/experimento/{metricas_python,comparacao_orange,efeito_vazamento,auc_univariada,sensibilidade}.csv` e `numeros.json`. Fluxos e capturas do Orange: `relatorio/semana4/{fluxo_correta,fluxo_vazamento}.ows` e `capturas/`. Decisões e alternativas descartadas: `relatorio/notas_para_defesa_semanas_2_4.md` (D44 a D52).